

VIP C2 PROGRAMME — FULL STACK DEVELOPMENT WITH
MERN

Full Stack Development (FSD) Documentation

Online Complaint Registration System — MERN Stack Complaint
Management Platform

Phase: Brainstorming & Ideation Phase

Document Version: 1.1

Date: 18th of june, 2026

Prepared by: YERRA JEEVANA

Classification: Project Documentation — For Academic Submission

1. INTRODUCTION

The **Online Complaint Registration System** is a comprehensive digital platform developed to modernize and streamline the grievance redressal process for civic and organizational environments. Built entirely on the MERN stack (MongoDB, Express.js, React.js, Node.js), the application addresses the inherent inefficiencies, lack of transparency, and communication gaps prevalent in traditional, manual complaint handling mechanisms.

The system features a robust, multi-tier architecture enabling distinct operational portals for Citizens, Service Agents, and Administrators. Key capabilities include secure JWT-based authentication, real-time bidirectional communication via Socket.IO, lifecycle tracking, automated assignment workflows, and analytics-driven administrative dashboards. Deployed on modern cloud infrastructure (Vercel for the frontend and Render for the backend), the project demonstrates a highly scalable, maintainable, and responsive engineering approach developed by Yerra Jeevana.

2. PROJECT OVERVIEW

The project is compartmentalized into independent logic modules. This developer-centric approach ensures code scalability and strict separation of concerns, providing dedicated interfaces tailored to the exact requirements of each user role.

2.1 Core Features

- **User Registration & Login:** Secure onboarding utilizing Bcrypt password hashing.
- **JWT Authentication:** Stateless sessions maintained via HTTP-only cookies.
- **Role-Based Access Control (RBAC):** Strict routing enforcement for Citizen, Agent, and Administrator roles.
- **Complaint Registration & Tracking:** End-to-end lifecycle monitoring from submission to resolution.
- **Complaint Assignment & Agent Dashboard:** Automated and manual assignment workflows.
- **Complaint Status Management:** Strict state machine transitions (Pending → Assigned → In Progress → Resolved).
- **Real-Time Chat & Notifications:** Instant Socket.IO powered bidirectional communication and push alerts.
- **Feedback & Ratings:** Post-resolution quality assurance metrics.
- **Analytics Dashboard:** High-level administrative insights and system performance tracking.

- **MongoDB Atlas Integration:** Cloud-native NoSQL data persistence.
- **Cloud Deployment:** Highly available distribution via Vercel and Render.

2.2 Modules

- **Citizen Module:** Handles account management, complaint lodging, historical tracking, and direct agent communication.
- **Agent Module:** Manages assignment pipelines, status adjustments, and citizen communications.
- **Admin Module:** Controls governance loops, verifying agents, and managing user access.
- **Complaint Management Module:** The central state engine processing all ticket data and file attachments.
- **Real-Time Chat Module:** Binds Socket.IO events to specific complaint ID rooms.
- **Notification Module:** Broadcasts lifecycle milestones to respective stakeholders.
- **Feedback Module:** Collects post-resolution metrics for analytics evaluation.
- **Analytics Dashboard Module:** Aggregates dataset metrics into visual charts for administrative review.

3. SYSTEM ARCHITECTURE

3.1 Frontend Architecture

Technology Stack: React.js, Vite, Material UI, Axios, React Router.

The client is structured as a Single Page Application (SPA). React Router manages navigation seamlessly without full-page reloads. Context API maintains global state for authentication sessions and real-time socket connections. Axios interceptors handle REST API communications centrally, automatically attaching authentication tokens and managing error responses. Material UI provides highly responsive, accessible presentation components.

3.2 Backend Architecture

Technology Stack: Node.js, Express.js, JWT Authentication, Socket.IO.

The backend operates on a strict Model-View-Controller (MVC) architectural pattern. Express.js routes HTTP traffic through custom middleware (for JWT validation, RBAC checks, and payload sanitization) to specialized controllers. The Socket.IO server runs concurrently alongside the Express application, processing WebSocket connection events and broadcasting targeted namespace messages.

3.3 Database Architecture

Technology Stack: MongoDB Atlas, Mongoose ODM.

MongoDB Atlas provides highly available, cloud-hosted document storage. The schema design leverages Mongoose ODM to enforce validation rules, default states, and relational constraints using ObjectIds. The persistence layer utilizes intelligent indexing strategies to optimize aggregation and read performance.

3.3 Database Architecture

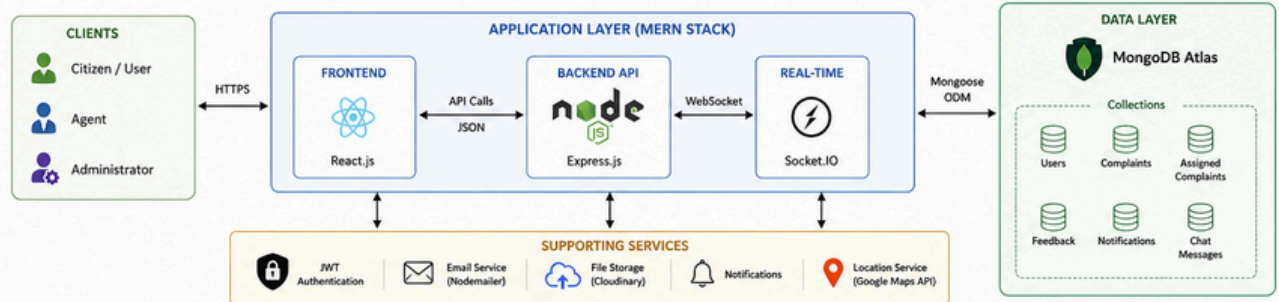


Figure 3.1 – Overall System Architecture

3.4 Deployment Architecture

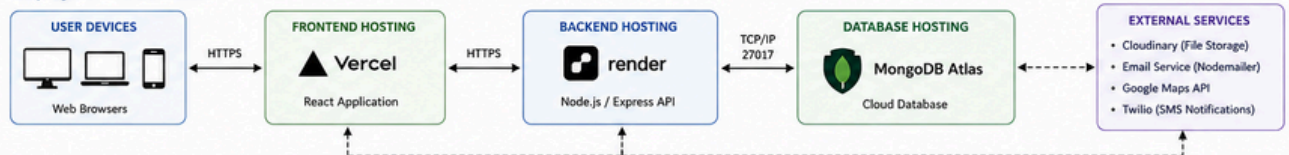


Figure 3.2 – Deployment Architecture

3.5 Real-Time Communication Architecture

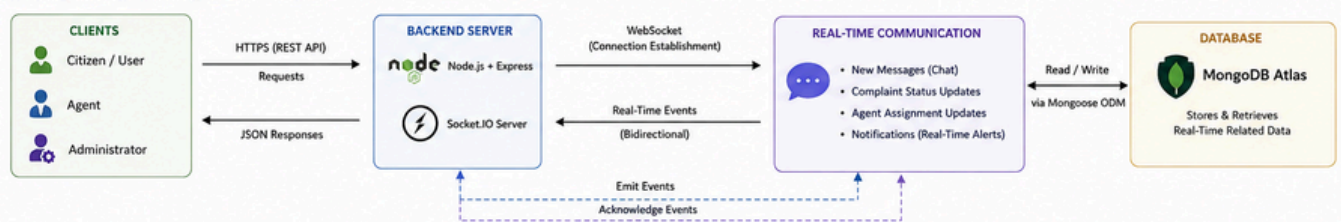


Figure 3.3 – Real-Time Communication Architecture

4 REQUIREMENT ANALYSIS

4.1 Functional Requirements

Req ID	Module	Description
FR-01	Authentication	System shall allow users to register and login using JWT.
FR-02	Access Control	System shall implement Role-Based Access Control (Citizen, Agent, Admin).

Req ID	Module	Description
FR-03	Complaints	Citizens shall be able to register and track complaints.
FR-04	Agent Ops	Admins can assign complaints; Agents can update statuses.
FR-05	Communication	System shall provide real-time chat via Socket.IO.
FR-06	Notifications	System shall push live notifications for status changes.
FR-07	Analytics	System shall provide graphical dashboard insights for Admins.
FR-08	Feedback	Citizens shall rate and review resolved issues.

4.2 Non-Functional Requirements

Req ID	Category	Description
NFR-01	Security	Passwords must be hashed using Bcrypt; APIs protected.
NFR-02	Performance	API response times should be under 500ms.
NFR-03	Scalability	The architecture must support horizontal scaling via decoupled MERN.
NFR-04	Usability	The interface must be fully responsive across all device viewports.
NFR-05	Availability	Hosted on Vercel and Render for 99% expected uptime.

5 DATABASE DESIGN

The persistence layer utilizes MongoDB Atlas for NoSQL document storage, managed programmatically through Mongoose. This ensures flexibility in evolving schemas while retaining structural integrity.

5.1 Database Schema Collections

Collection	Field Name	Data Type	Constraints / Description
User	_id	ObjectId	Primary Key
	name	String	Required
	email	String	Required, Unique
	password	String	Required, Encrypted
	role	String	Enum: Citizen, Agent, Admin
	isApproved	Boolean	Default: false for Agent
Complaint	_id	ObjectId	Primary Key
	userId	ObjectId	Foreign Key (User)
	category	String	Required
	status	String	Enum: Pending, Assigned, In Progress, Resolved
	address, city, pincode	String	Location data strings
	assignedAgentId	ObjectId	Foreign Key (User - Agent)
	attachments	Array	URLs of uploaded images
Message	_id	ObjectId	Primary Key
	complaintId	ObjectId	Foreign Key (Complaint)
	senderId	ObjectId	Foreign Key (User)
	text	String	Content of message
Notification	_id	ObjectId	Primary Key
	recipientId	ObjectId	Foreign Key (User)
	message	String	Content

Collection	Field Name	Data Type	Constraints / Description
	isRead	Boolean	Default: false
Feedback	_id	ObjectId	Primary Key
	complaintId	ObjectId	Foreign Key (Complaint)
	rating	Number	Min 1, Max 5
	review	String	Optional feedback text

5.2 Entity Relationship Diagram

FIGURE 5.1 – ENTITY RELATIONSHIP DIAGRAM

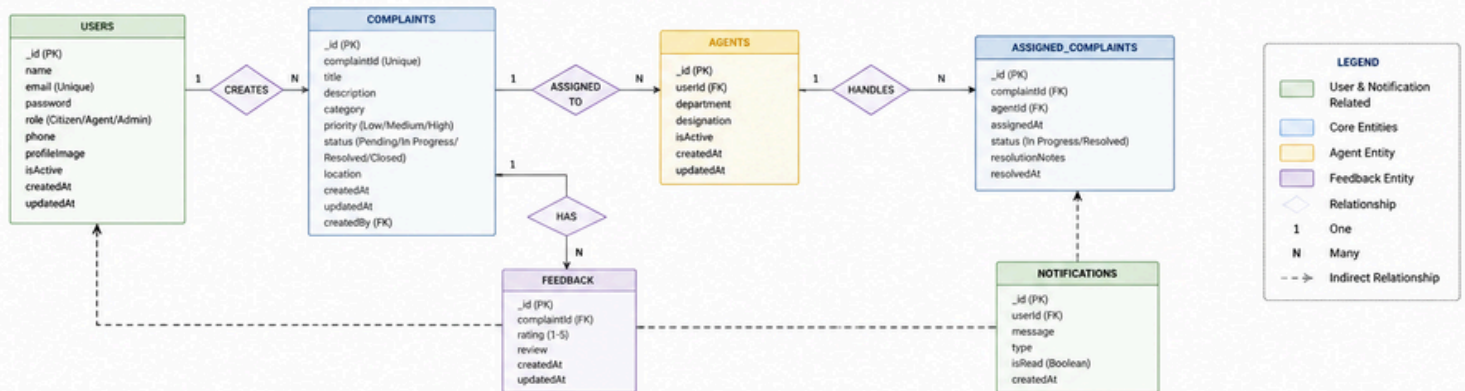


FIGURE 6.1 – USE CASE DIAGRAM

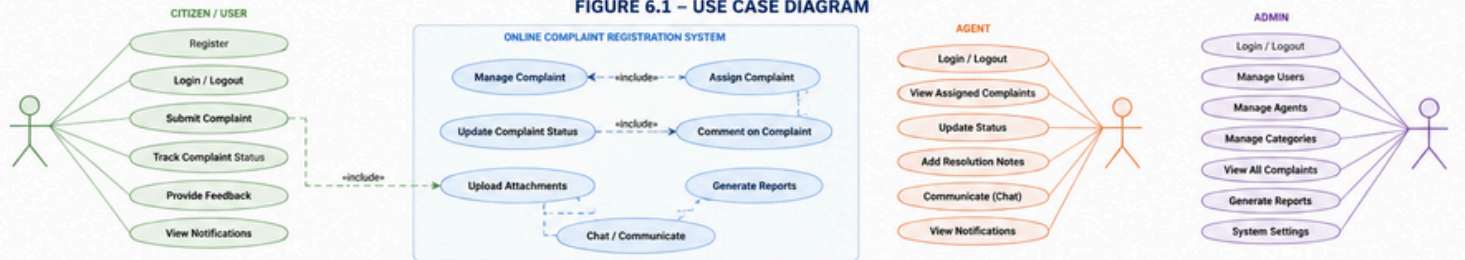
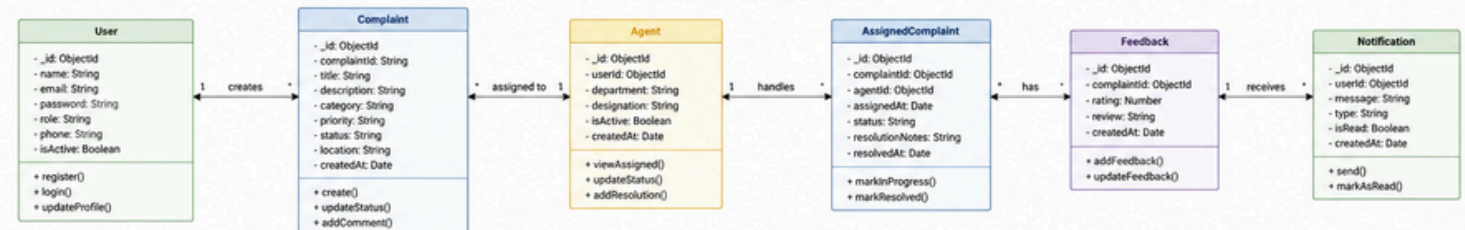
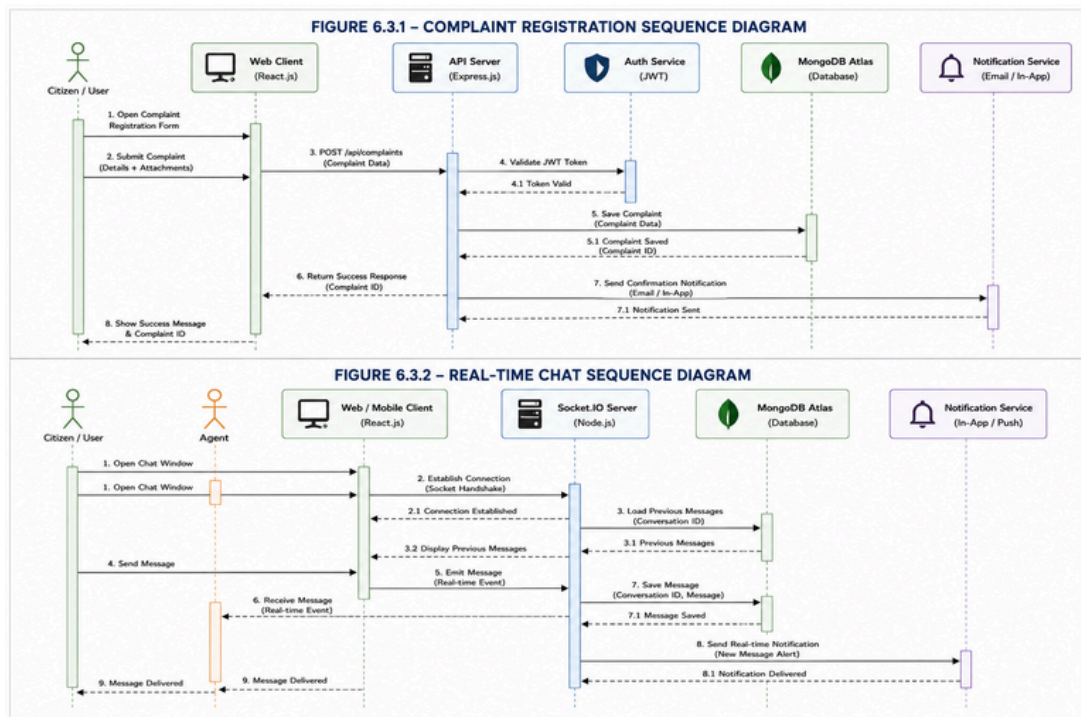


FIGURE 6.2 – CLASS DIAGRAM



6.3 Sequence Diagrams



7 MODULE DESIGN

The system is compartmentalized into independent logic modules. This developer-centric approach ensures code scalability.

- **Citizen Module:** Handles account management, complaint lodging, historical tracking, and feedback.
- **Agent Module:** Manages assignment pipelines, status adjustments, and citizen communications.
- **Admin Module:** Controls governance loops, verifying agents and analyzing system metadata.
- **Real-Time Chat Module:** Binds Socket.IO events to specific complaint ID rooms.
- **Notification Module:** Automates state-change alerts.
- **Feedback Module:** Collects post-resolution metrics for analytics.

8.IMPLEMENTATION

8.1 Setup Instructions

Software Requirements: Node.js (v18+), npm, MongoDB Atlas cluster, Git.

Environment Variables:


```
# Server (.env)
PORT=5000
MONGO_URI=mongodb+srv://<db_user>:<db_pass>@cluster.mongodb.net
JWT_SECRET=super_secret_jwt_key
JWT_EXPIRES_IN=30d
NODE_ENV=development

# Client (.env)
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
```

8.2 Folder Structure

```
Client
├── assets
│   └──
components
├── context
│   └── services
Server
├── config
│   └──
controllers
├──
middleware
├── models
├── routes
└── utils
```

8.3 Running the Application

Backend: npm install followed by npm start.

Frontend: npm install followed by npm run dev.

9 API DOCUMENTATION

9.1 Authentication APIs

Endpoint	Method	Description	Auth Req
/api/auth/register	POST	Register user. Req: name, email, password, role.	No
/api/auth/login	POST	Login. Res: JWT Cookie.	No
/api/auth/logout	POST	Clear session HTTP-only cookie.	Yes

9.2 Complaint APIs

Endpoint	Method	Description	Auth Req
/api/complaints	POST	Create Complaint. Req: category, description.	Yes (Citizen)
/api/complaints/my	GET	Get user complaints.	Yes (Citizen)
/api/complaints/:id/status	PUT	Update Status.	Yes (Agent/Admin)
/api/complaints/:id	GET	Track specific complaint details.	Yes

9.3 Admin APIs

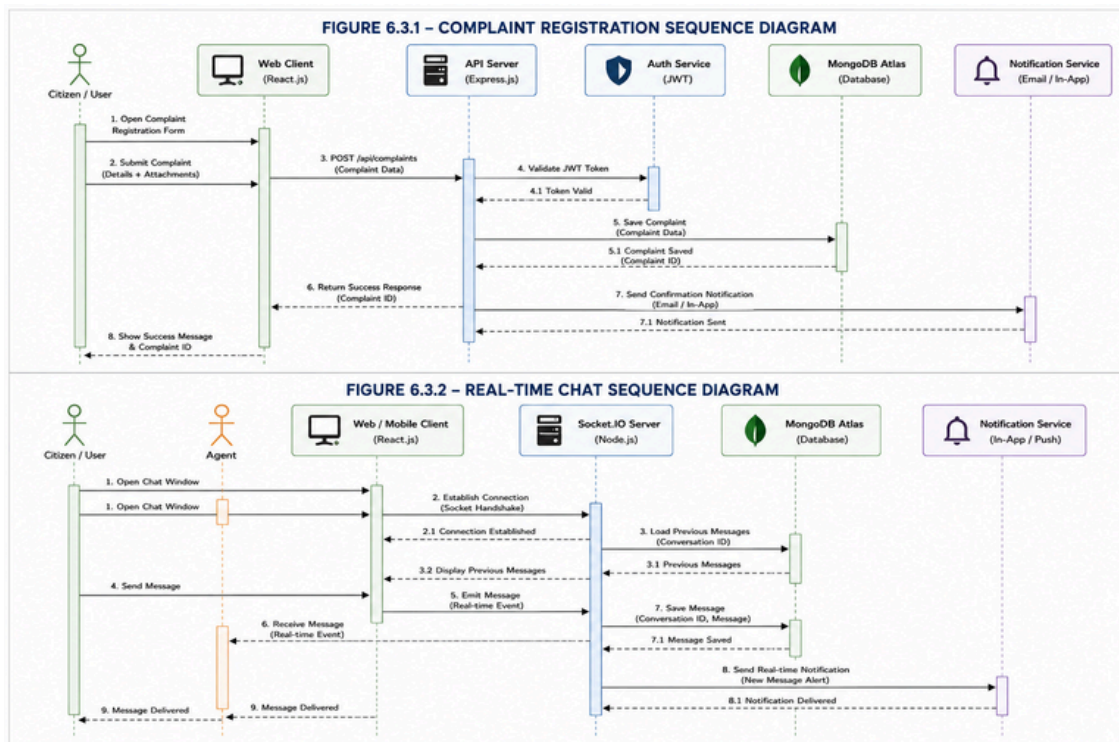
Endpoint	Method	Description	Auth Req
/api/admin/approve/:id	PUT	Approve agent account status.	Yes (Admin)
/api/admin/stats	GET	Dashboard aggregate stats.	Yes (Admin)
/api/admin/users	GET	Fetch user management grid.	Yes (Admin)
/api/admin/assign/:id	PUT	Assign agent to complaint.	Yes (Admin)

9.4 Chat, Feedback & Notification APIs

Endpoint	Method	Description	Auth Req
/api/messages	POST	Send REST fallback message.	Yes
/api/messages/:id	GET	Get complaint chat history.	Yes
/api/feedback	POST	Submit resolution feedback.	Yes (Citizen)
/api/notifications	GET	Get active user alerts.	Yes

10 AUTHENTICATION & AUTHORIZATION

The Developer integrated a stateless JWT architecture. Upon valid login via the authController, a token is signed and injected into an HTTP-only cookie to prevent XSS. Express middleware validates this token before fulfilling protected routes. The Role-Based Access Control (RBAC) ensures administrative and agent routes strictly reject unauthorized roles with a 403 Forbidden status.



11 REAL-TIME COMMUNICATION

Real-time updates are driven by Socket.IO. The implementation establishes a persistent WebSocket connection upon mounting the React client. Chat rooms are dynamically joined using the complaintId, ensuring isolated and secure message broadcasting between the specific citizen and the assigned agent without server polling overload.

12 TESTING

The testing phase validates functional integrity.

- **Unit Testing:** Code-level validation of utility functions, reducers, and Mongoose schemas.
- **Integration / API Testing:** Verifying REST endpoint security, payload schemas, and HTTP status codes via Postman.
- **Socket.IO Testing:** Validation of namespace connections and event broadcasting payloads.
- **User Acceptance Testing (UAT):** Manual workflow traversal simulating standard operational actions.
- **Deployment Testing:** Validating CORS, environment injections, and cloud database bridging.

13 USER INTERFACE

The frontend interface utilizes Material UI for a polished, highly responsive academic design. Below are the functional view spaces reserved for documentation.

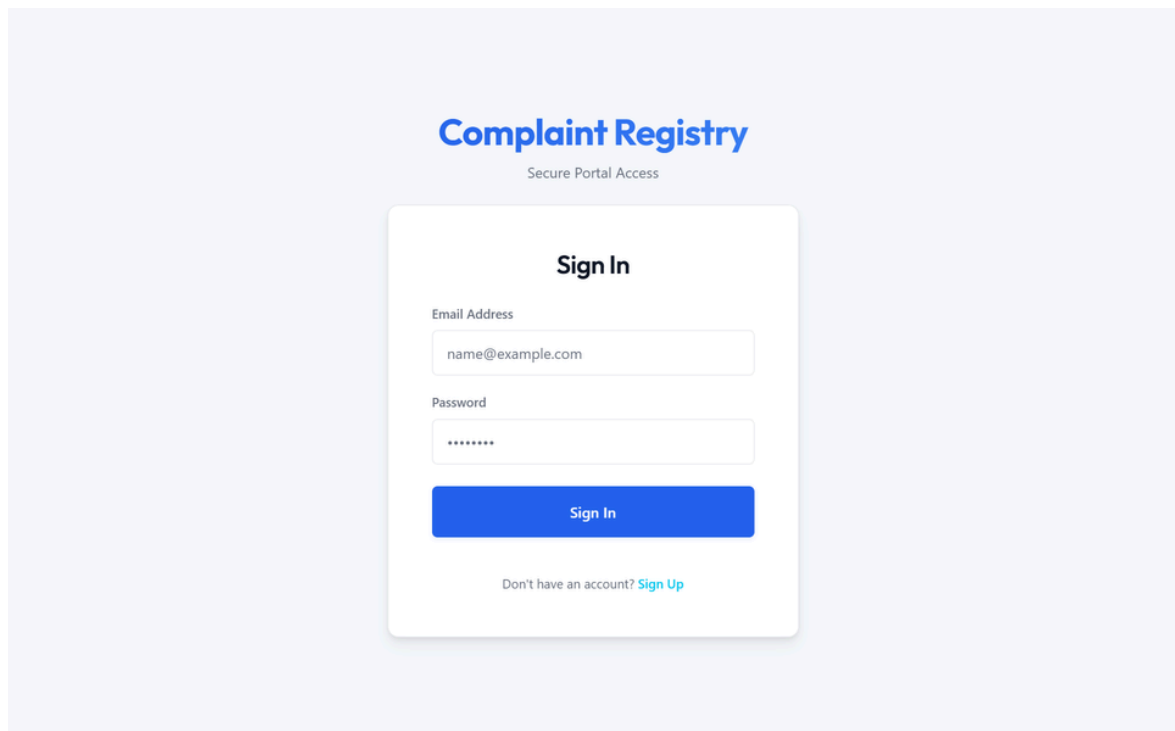


Figure 13.1 – Login Page

Complaint Registry

Create your secure gateway account

Sign Up

Full Name

John Doe

Email Address

name@example.com

Phone Number

9876543210

Password

Min 6 characters

I am registering as an:

☒ Citizen ☐ Field Agent (Officer)

Sign Up

Figure 13.2 – Registration Page

Complaint Registry Dashboard Lodge Complaint

Notifications

Mark all read

New Chat Message

suresh: "hello sir"

6/18/2026 01:29 PM

Max Ordinary Sign Out

My Complaints

Lodge, track progress, and review your registered grievances

+ Lodge a Complaint

TOTAL LODGED1

PENDING0

IN PROGRESS1

RESOLVED0

Search subject, category or comment...

AllPendingIn ProgressResolvedRejected

Unclean Roads

In Progress

Municipal

Filed on: Jun 18, 2026

Location: Hyderabad, Telangana

Unclean Roads need immediate response

Figure 13.3 – Citizen Dashboard

Complaint Registry
Dashboard
Lodge Complaint

Max
Ordinary
Sign Out

Lodge a New Complaint

Please provide precise information about the grievance so that our administration can assign the correct field technician.

Complaint Title / Subject

Complaint Category

Pincode (6-digit)

Detailed Comment / Description

Grievance Address / Location

City

State

Lodge Complaint

Figure 13.4 – Complaint Registration Form

Complaint Registry
Dashboard
Lodge Complaint

Joy
Ordinary
Sign Out

[← Back to Dashboard](#)

TRACKING TIMELINE

1

Submitted

2

Assigned

3

In Progress

4

Resolved

OTHER
Resolved

Improper roads Uppal to Ghatkesar

Detailed Grievance / Comment

need faster response

Grievance Location

Uppal to Ghatkesar road
Hyderabad, Telangana - 500088

Lodged On

Jun 17, 2026, 5:15 PM

Reopen Complaint
Submit Feedback & Star Rating

TASK MANAGEMENT

Grievance resolved successfully.

Reopen History

Total Reopens: 0 / 3

Figure 13.5 – Complaint Tracking Interface

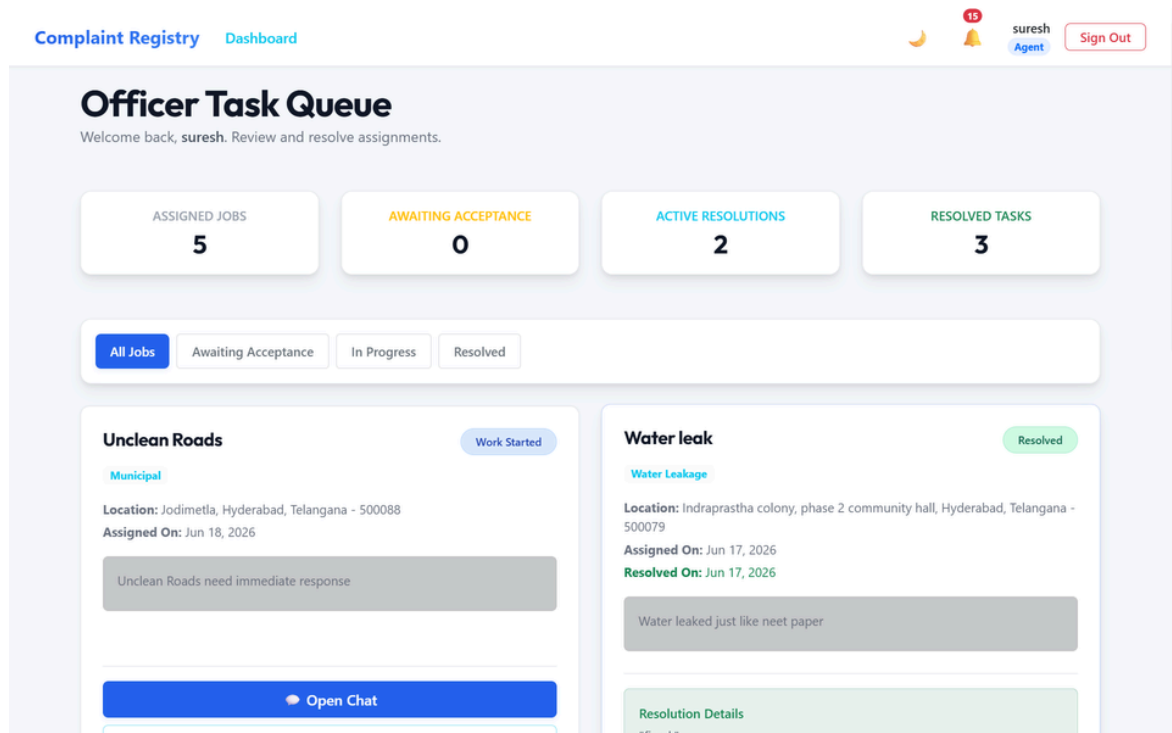


Figure 13.6 – Agent Dashboard

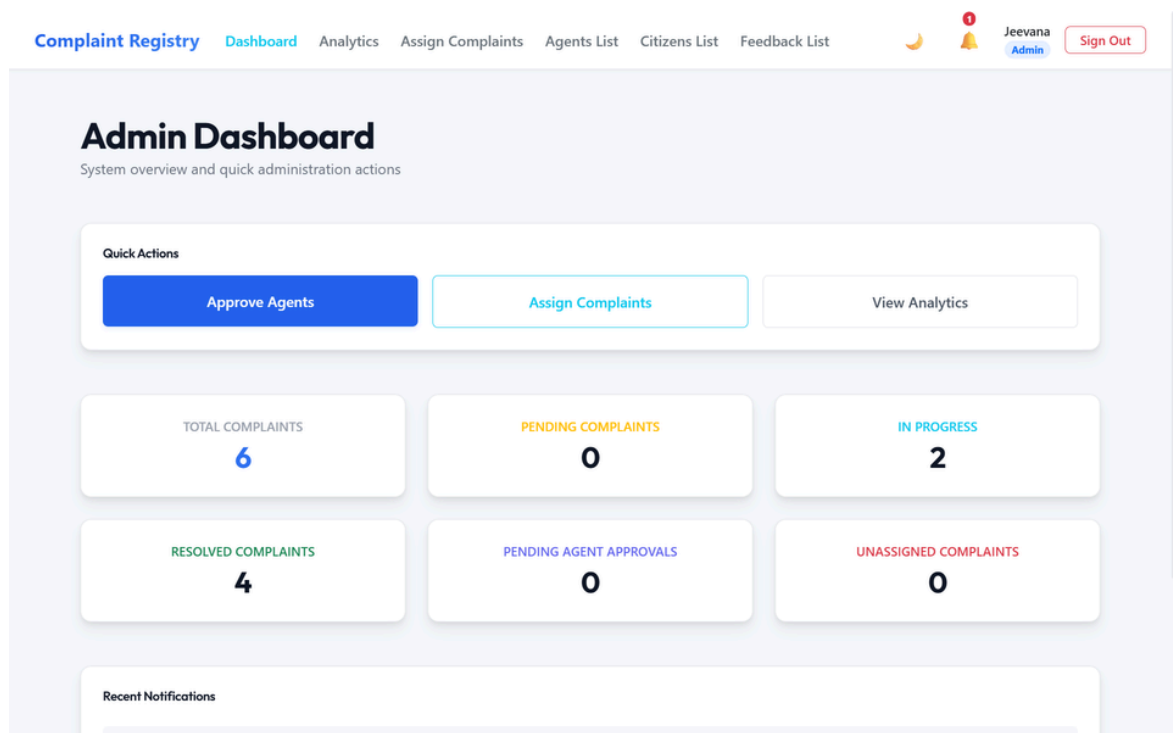


Figure 13.7 – Admin Dashboard

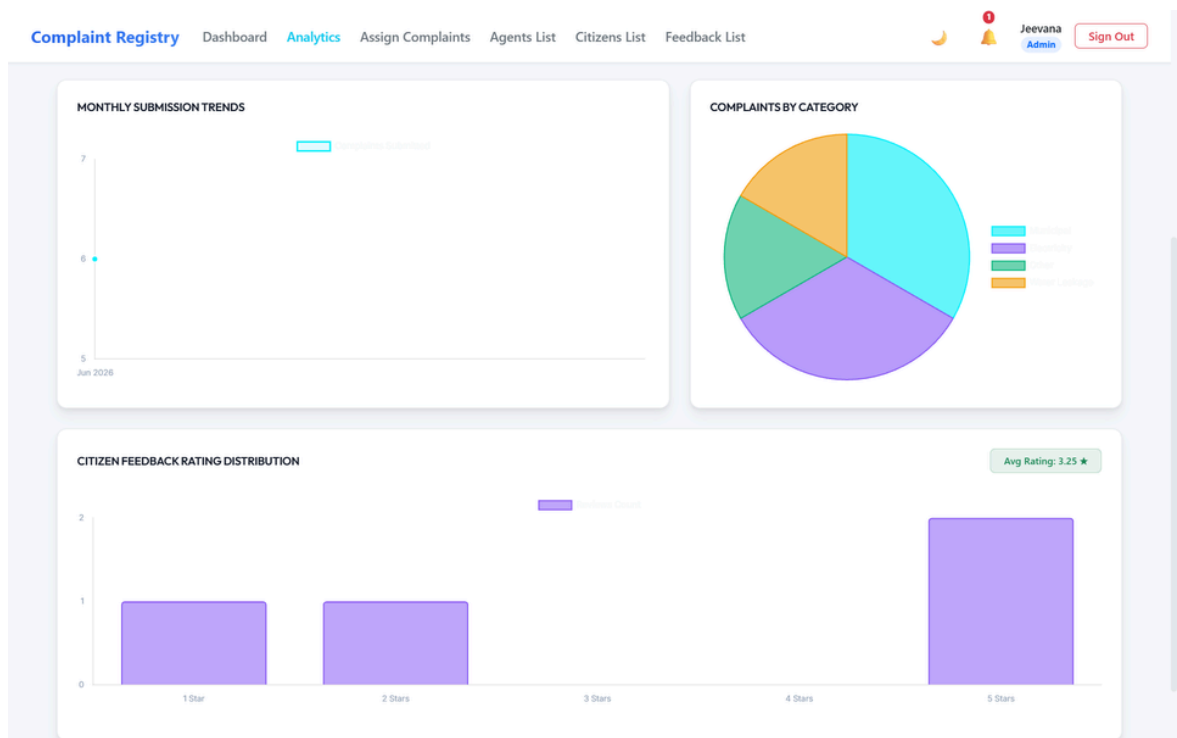


Figure 13.8 – Analytics Dashboard

The Feedback & Ratings Module allows users to provide feedback on their complaint resolution. The form includes the following elements:

- Header:** "Share Your Review" with a sub-header "How satisfied are you with the resolution of ticket #F7AC4E?".
- Overall Rating:** A section with five blue stars and a "Score: 5 / 5" indicator.
- Comments & Suggestions:** A text area for providing details about the resolution efficiency, agent conduct, etc., with a character count of "0 / 500 characters".
- Submit Review:** A prominent blue button to submit the feedback.

Figure 13.11 – Feedback & Ratings Module

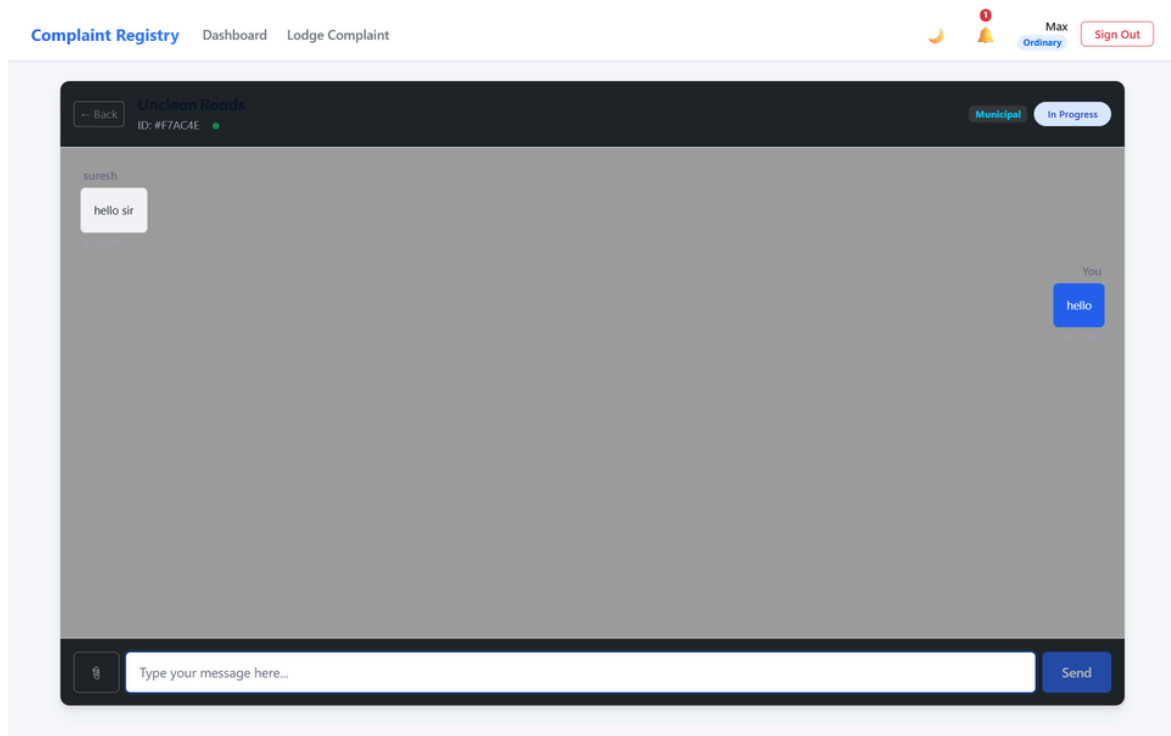


Figure 13.9 – Real-Time Chat System

14 DEPLOYMENT

The Developer utilized a decoupled cloud distribution strategy. Vercel hosts the React.js SPA, ensuring low-latency CDN delivery and continuous deployment upon git push. Render instances process the Express.js API, handling compute-heavy JWT checks and Socket traffic. Both environments strictly utilize hidden environment variables and integrate seamlessly with the Atlas Database.

15 RESULTS & DISCUSSION

The system successfully met all defined academic and practical objectives. The centralized architecture eliminates traditional processing delays, achieving end-to-end transparency. Socket.IO integration demonstrated minimal latency during inter-role chat testing, and the RBAC middleware strictly protected administrative routes as designed.

16 KNOWN ISSUES

- **Render Free Tier Cold Start:** API endpoints may sleep after 15 minutes of inactivity, causing initial requests to delay by 30-50 seconds.
- **Socket.IO Reconnection Handling:** Minor connection drops occur when switching cellular networks; mitigated via auto-reconnect configurations.
- **Network Latency:** Image attachment uploads are susceptible to local bandwidth drops.
- **Internet Dependency:** The platform relies heavily on consistent internet connectivity for real-time WebSocket features.
- **Browser Compatibility Constraints:** Older versions of Safari may restrict certain strict cross-site cookie behaviors.

17 FUTURE ENHANCEMENTS

Feature	Description
AI Complaint Categorization	NLP algorithm to auto-assign departments based on descriptions.
Mobile Application	React Native app development for native push alerts.
SMS/Email Notifications	Integration with Twilio and Nodemailer for offline alerts.
Geo-Tagged Complaints	Map integration for pinpointing infrastructure damage coordinates.
Image Recognition	Automated visual sorting of complaint attachments.
Predictive Complaint Analysis	Machine learning models forecasting seasonal grievance spikes.
Agent Performance Analytics	Granular resolution speed and satisfaction rating metrics.
Multi-Language Support	i18n implementation for wider accessibility.

18 CONCLUSION

The Online Complaint Registration System, developed by Yerra Jeevana, serves as a comprehensive validation of Full Stack Development principles using the MERN stack. By establishing a digital-first, real-time platform, the project systematically resolves the inefficiencies of traditional grievance workflows. With secure deployment, robust architecture, and clear pathways for future AI enhancements, the system stands as a scalable, production-ready enterprise solution.

19 REFERENCES

- React.js Official Documentation: <https://reactjs.org/>
- Express.js Framework Guide: <https://expressjs.com/>
- MongoDB Atlas Manual: <https://docs.atlas.mongodb.com/> •
- Socket.IO Implementation Docs: <https://socket.io/docs/v4/> •
- JWT Security Standards (RFC 7519): <https://jwt.io/>

20 APPENDICES

The appendices (reserved for final bound submission) contain raw schema configurations, package.json dependencies lists, sample dummy data seeding scripts, and testing log outputs utilized by the developer throughout the lifecycle of the project.